AI Lab Assignment 2: Tic-Tac-Toe Game

Minimax Algorithm with Alpha-Beta Pruning • Intelligent Agent Design

1. Aim

Design an intelligent agent that plays Tic-Tac-Toe by choosing moves using the Minimax algorithm, with Alpha-Beta Pruning to avoid exploring branches that cannot affect the final decision.

2. Board and Symbols

Board consists of 9 cells. Blank = space, Human = X, AI = 0. User enters positions 1–9; internal Python indexes are 0–8.

```
1 | 2 | 3
---------
4 | 5 | 6
---------
7 | 8 | 9
```

`board = [" "] * 9` creates cells. `import math` provides `math.inf`.

3. display() & 4. winner(b)

display(): Prints 3×3 board layout using row index loops and `" | ".join(board[i:i+3])`.

winner(b): Checks all 8 winning combinations (3 rows, 3 columns, 2 diagonals). Returns X or 0 if winning, otherwise None.

5. Minimax Algorithm

Minimax explores possible future moves. **O (AI)** maximizes score; **X (Human)** minimizes score.

```
      if result == "O": return 10 - depth if result == "X": return depth - 10 if " " not in b:
return 0
```

Scoring: AI win yields positive score (quicker wins score higher via smaller depth); Human win yields negative score (delaying loss makes it less negative); Draw yields 0.

6. Alpha-Beta Pruning

Speeds up search without altering results.

- **Alpha (α):** Best score max player can guarantee.
- **Beta (β):** Best score min player can guarantee.
- **Prune Check:** `if beta <= alpha: break`

7. ai_move() & 8. Game Loop

ai_move(): Tests placing 0 in each empty cell, runs Minimax, resets cell, and plays the optimal move index + 1.

Main Loop: Prompts user input, validates range/occupancy, checks wins/draws alternatively until game ends.

9. Expected Behavior & 10. Short Viva Script

> **Short Viva Script:** "Ma'am, this program creates a Tic-Tac-Toe game where human plays X and AI plays O. Minimax simulates future outcomes: AI maximizes score and human minimizes it. Alpha-Beta Pruning uses alpha and beta bounds to skip irrelevant branches. The AI evaluates legal moves and selects the highest scoring choice."

11. Likely Viva Questions & 12. Quick Revision

Question	Answer
What is Minimax?	A recursive game-search algorithm choosing optimal moves assuming both play perfectly.
What does Alpha-Beta Pruning do?	Skips search branches that cannot affect the decision, preserving the optimal result.
Why use depth in score evaluation?	To prefer quicker wins and delay defeats when losing.
Quick Revision:	X = Human/Minimizer | O = AI/Maximizer | Prune when β ≤ α | Positive = AI win.